KAM-Net: Kilobyte-Scale Ultralightweight Attention-Based Network for Glass Defect Detection With Algorithm/Hardware Co-Design

Bingrui Zhao, *Student Member, IEEE*, Yaonan Wang, *Member, IEEE*, Hui Zhang, *Member, IEEE*, Jiaxuan Liu, and Jinzhou Zhang

***Abstract*—Glass manufacturing processes often yield complex defects such as bubbles, stones, and scratches. While machine vision systems powered by deep learning offer effective defect detection, real-time inference on production lines remains a hurdle. To address this, we propose KAM-Net, an ultralightweight kilobyte-scale defect detection model utilizing depthwise separable convolution (DSC) and improved triplet attention (TA). With just 4.2 kB of parameter storage, KAM-Net achieves impressive performance, achieving a 92.3% mIOU and 91.5% *F*1-score on the glass defect dataset, surpassing traditional models like UNet. It also demonstrates competitive performance in public classification datasets. Then, we design an efficient algorithm/hardware codesign for KAM-Net on field programmable gate array (FPGA). Leveraging parallelism and pipeline techniques, KAM-Net achieves a remarkable processing speed of 2564 FPS, outperforming GPU-based processors. Notably, our work marks the first hardware implementation of the TA mechanism, showcasing its practical application potential. Our detection system ensures high-precision defect identification, matching real-time image acquisition speeds of production lines.**

***Index Terms*—Attention mechanism, convolutional neural network (CNN) acceleration, defect detection, field programmable gate array (FPGA), machine vision.**

I. Introduction

GLASS plays a pivotal role across a spectrum of industrial sectors, encompassing electronic displays, packaging, and photovoltaic installations. However, the intricate nature of glass manufacturing processes invariably gives rise to undesirable imperfections such as scratches, bubbles, and impurities, as shown in Fig. 1(a). Historically, the detection of such defects relied heavily on manual inspection, a labor-intensive and subjective endeavor fraught with inconsistencies in standards. Recent advancements in optical and computational technologies have paved the way for the integration of machine vision solutions in defect detection tasks. A conventional approach involves capturing images using cameras stationed along the production line, followed by automated analysis to identify and locate defects. This detection process furnishes critical insights for subsequent quality control protocols.

In the past few years, many works on glass defect detection systems have been reported. Zhou et al. [1] applied a visual attention model and wavelet transform to detect defects in glass bottle bottoms. Jiang et al. [2] built a codec-type deep neural network to detect defects in mobile phone screen glass. Compared with traditional image processing algorithms, the average recall rate has increased by 4.6%. Yang et al. [3] also proposed a two-stage cascaded glass surface defect detection method based on deep learning. First, the feature extraction of glass samples was performed through the YoloV3 model, and then, the U-Net model was used to segment the defective parts. Their defect detection scale can reach a minimum of 26 μm, which meets the detection requirements of most glass surface defects. From the above related work, the glass defect detection algorithm based on deep learning is claimed to be able to surpass the accuracy of traditional image algorithms and is gradually favored by researchers.

While industrial machine vision systems prioritize detection accuracy, it is essential to ensure that detection speed aligns with the production line's pace. Typically, in machine vision defect detection setups, the imaging and image processing systems (PSs) are physically segregated. The imaging components, like the light source and camera, are positioned near the product on the assembly line to capture clear surface images. Conversely, the image PS often operates separately, relying on communication interfaces to obtain images for defect detection. However, as image sensor capabilities improve, the scale of acquired images increases. Bandwidth limitations can cause transmission times to exceed image processing durations. Integrating the imaging and PSs to enable edge-based detection can significantly reduce time overheads. Therefore, researchers began to study lightweight detection algorithms and their implementation on edge chips [4], [5], [6]. Related research has pointed out that field programmable gate arrays (FPGAs) are the most promising platform for implementing convolutional neural network (CNN) algorithms in edge-side applications, due to factors such as performance, power con-

Received 15 March 2025; revised 29 June 2025; accepted 16 July 2025. Date of publication 12 August 2025; date of current version 15 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62027810 and Grant 92148204, in part by the National Key Research and Development Program of China under Grant 2021ZD0114503, in part by Hunan Leading Talent of Technological Innovation under Grant 2022RC3063, in part by the Top Ten Technical Research Projects of Hunan Province under Grant 2024GK1010, and in part by the Key Research and Development Program of Hunan Province under Grant 2023GK2068 and Grant 2022GK2011. The Associate Editor coordinating the review process was Dr. Pawel Kwiatkowski. *(Corresponding author: Hui Zhang.)*

Bingrui Zhao, Yaonan Wang, Hui Zhang, and Jiaxuan Liu are with the National Engineering Research Center of Robot Visual Perception and Control Technology, School of Robotics, Hunan University, Changsha, Hunan 410082, China (e-mail: neuzhaobr@163.com; yaonan@hnu.edu.cn; zhanghuihby@126.com; ljx9339@hotmail.com).

Jinzhou Zhang is with the School of Electrical and Information Engineering, Changsha University of Science and Technology, Changsha, Hunan 410114, China (e-mail: zhangjinzhouhn@126.com).

Digital Object Identifier 10.1109/TIM.2025.3593602

1557-9662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

sumption, cost, and reprogrammability [7], [8]. Lucia et al. [9] presented an FPGA-based real-time SURF feature detection architecture using separable convolution and pipelined design, achieving 140-Mpixels/s processing speed with only 20% resource utilization, suitable for multiresolution video streams. Cížek and Faigl [10] optimized model predictive control (MPC) implementation on FPGAs via high-level synthesis (HLS), combining automatic code generation and algorithm optimization to ensure real-time performance while reducing resource consumption, outperforming traditional microcontroller solutions. Li et al. [11] proposed binary quantization of part of the network layers in CEDNet for the task of scene text recognition, which greatly reduced the storage and calculation burden of the model when it was running while ensuring the detection accuracy and designed a high-level pipeline for the algorithm model optimized implementation. Lyu et al. [12] designed a driving area detection network ChipNet using radar data, using quantization and hole convolution to reduce the model size, and implemented it in FPGA. The abovementioned software–hardware collaborative acceleration work not only made lightweight improvements to the algorithm but also used chips for hardware acceleration, which provided us with inspiration. However, edge devices often feature low power consumption and low computing power, which may not be sufficient to support large-scale image processing requirements, especially computationally intensive operations such as convolution operations that are common in deep learning. Therefore, it is necessary to design a corresponding hardware implementation scheme for the algorithm.

In this article, in order to implement a high-accuracy real-time machine vision online inspection system for glass defect detection on high-speed production lines, we propose an FPGA accelerator for KAM-Net. The accelerator can be embedded in a production line camera and directly output semantic segmentation image information containing defects in the image, as shown in Fig. 1(b). In summary, this article makes the following contributions.

1) An ultralightweight defect detection model KAM-Net is proposed. This model can reduce the computational burden while ensuring detection accuracy. The proposed model requires only 4.2 kB of parameter storage space to achieve 92.5% mIOU and 91.9% $F1$-score in the glass defect detection dataset. It also demonstrates competitive performance in public classification datasets like the Tile Defect dataset [26].
2) We introduced an improved lightweight triplet attention (TA) mechanism and successfully implemented it on an efficient FPGA platform. To our knowledge, this marks the first implementation of the TA mechanism as a dedicated hardware circuit, demonstrating its practical viability for on-chip applications.
3) We present a dedicated FPGA hardware accelerator for KAM-Net leveraging pipeline technology. Our approach includes the design of a dedicated modular hardware accelerator and its integration into a heterogeneous embedded system tailored for KAM-Net. This involves the development of various hardware circuits, including a row buffer-based uninterrupted convolution hardware accelerator. Our KAM-Net hardware system achieves an exceptional processing speed of 2564 FPS on the Xilinx ZCU102 platform.

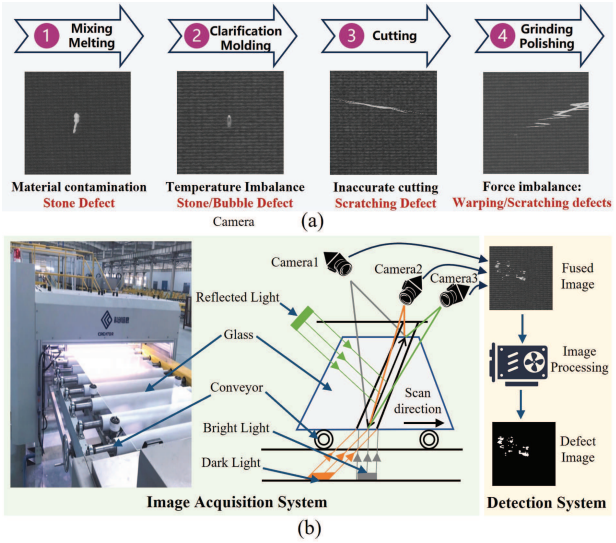


Fig. 1. (a) Manufacturing process of glass with potential defects. (b) Glass image acquisition and defect detection system.

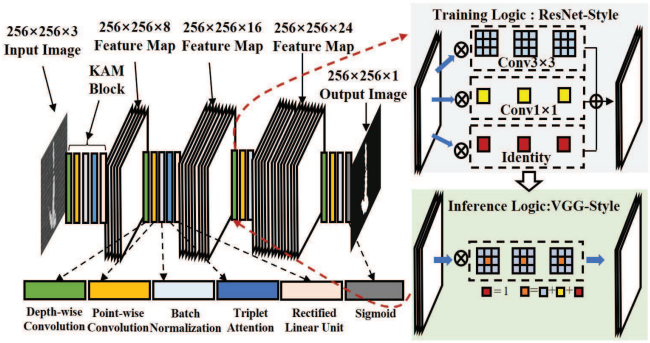


Fig. 2. Overall architecture of ultralightweight KAM-Net.

The rest of this article is organized as follows. Section II discusses the glass defect detection network. Then, Section III presents the hierarchical design of the FPGA accelerator. Section IV discusses the performance results of the detection system. Finally, Section V concludes this article.

II. Algorithm Design

As deep learning progresses, researchers increasingly develop complex models for higher accuracy in image processing. However, this poses challenges for platforms with limited computing power, such as FPGA. For lightweight, low-power, and high-performance solutions, we introduce KAM-Net, an ultralightweight neural network for glass surface defect detection. This network combines depthwise separable convolution (DSC) and a TA mechanism for efficient and accurate defect segmentation with minimal parameter usage.

A. KAM-Net Block

The KAM-Net architecture, illustrated in Fig. 2, consists of four blocks, each employing DSC for its convolutional

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

layer. DSC divides standard convolution into depthwise and pointwise stages, significantly reducing parameters and making the network lightweight [13]. Batch normalization (BN) layers [14] follow each pointwise convolutional layer for model fitting. Despite parameter reduction, DSC may limit information interaction between channels. To address this, we introduce TA [15] in the first two blocks to enhance information interaction within and between feature maps. TA is a parameterless attention mechanism detailed in Section II-B of Section II. ReLU activation functions [16] are added after TA for training stability and nonlinear representation capabilities.

Unlike typical DSC blocks, we introduce reparameterization within each depthwise convolution [17], as depicted on the right of Fig. 2. Reparameterization treats depthwise convolution as a residual module during training, where its input simultaneously enters the 3×3 convolution channel, 1×1 convolution channel, and identity channel. During inference, the parameters of the 1×1 convolution channel and the identity channel are mapped to those of the 3×3 convolution channel [18] leveraging convolution operation additivity. Reparameterization allows for residual module construction during training, potentially enhancing model classification performance compared to single-path direct connection architectures [19]. During model inference, the equivalent convolutional layer inherits the performance benefits of the residual module while maintaining real-time computational efficiency.

Additionally, to further reduce the computational overhead during inference, we employed BN folding techniques [20], enabling the parameters of the BN layer to be effectively integrated into the convolutional layer during inference, thus minimizing computational load during the inference process. For a convolutional layer of kernel size K, input channel dimension C_{in}, and output channel dimension C_{out}, the weight matrix is denoted as $\boldsymbol{W}' \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times K \times K}$ and bias is denoted as $\boldsymbol{b}' \in \mathbb{R}^{D}$. A batchnorm layer contains accumulated mean $\boldsymbol{\mu}$, accumulated standard deviation $\boldsymbol{\sigma}$, scale $\boldsymbol{\gamma}$, and bias $\boldsymbol{\beta}$. Since convolution and batchnorm at inference are linear operations, they can be folded into a single convolution layer with weights $W = \boldsymbol{W}' \times (\boldsymbol{\gamma}/\boldsymbol{\sigma})$ and bias $b = (\boldsymbol{b}' - \boldsymbol{\mu}) \times (\boldsymbol{\gamma}/\boldsymbol{\sigma}) + \boldsymbol{\beta}$.

To balance hardware efficiency and segmentation performance across tasks of varying complexity, the KAM-Net architecture is designed to be modular and scalable. Specifically, the network is composed of stacking multiple KAM blocks, whose number can be flexibly adjusted based on target application scenarios. In our implementation, the base configuration of KAM-Net with four blocks is adopted for real-time glass defect detection on resource-constrained FPGAs. For more complex public datasets, a deeper version with six KAM blocks is employed to enhance segmentation capability while still maintaining compactness and efficiency. This scalable design highlights the versatility of KAM-Net across different deployment requirements.

B. Optimized HFTATA Module

With the rise of deep learning, attention mechanisms have become crucial for improving image classification performance, allowing models to focus on important regions or features during processing. We adopt the TA mechanism for two reasons. First, it has minimal parameters, reducing memory demands on FPGA platforms. Second, it operates across channel, row, and column dimensions, partially compensating for accuracy degradation in depth-separable convolution. After evaluating tradeoffs between accuracy and complexity, we integrate TA into the first two KAM-Net blocks, as depicted by the blue box in Fig. 2. The original computation process of the TA mechanism, illustrated in Fig. 3, involves three parallel operations on an input tensor with C channels, W width, and H height. The first two sets of channels facilitate cross-dimensional information interaction (channel attention) between channel C and spatial dimensions W and H, while the last set of channels addresses spatial attention between dimensions W and H. Each set of channels includes ZPool, convolution, and sigmoid operations. To streamline computations, we proposed the hardware-friendly triplet attention (HFTA). We reduce the convolution kernel size to 1×1 to minimize the number of DSPs required in hardware implementation. Additionally, we replace the conventional sigmoid activation function, which involves exponential operations and is computationally intensive, with the more hardware-efficient hard sigmoid (Hsigmoid). The original sigmoid function is defined as

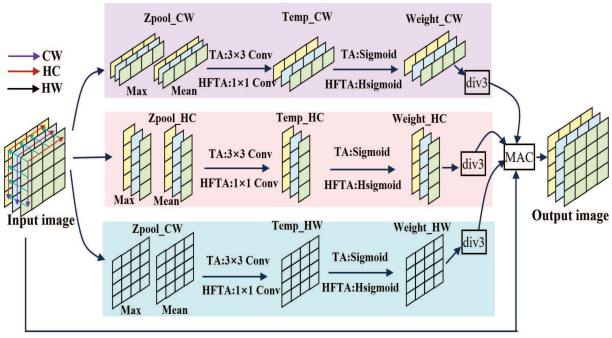


Fig. 3. TA and the proposed hardware-friendly version (HFTA).

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}. \tag{1}$$

While effective in modeling nonlinearities, the exponential calculation in (1) imposes a heavy burden on resource-constrained platforms such as FPGAs. To address this, we adopt the Hsigmoid function, a piecewise linear approximation that significantly reduces computational complexity

$$H\text{sigmoid}(x) = \frac{\min\left(\max\left(x + 3, 0\right), 6\right)}{6}. \tag{2}$$

This approximation eliminates the need for exponentials and the divide by 6 can be replaced by multiplication, making it more suitable for efficient implementation on FPGA hardware.

C. Overall Parameter Details

The parameters of KAM-Net are shown in Table I. It is worth mentioning that our network only contains 1086 model parameters in total. If the parameters are stored with 32-bit floating-point numbers, only 4.2 kB of storage space is required. Compared with classic models such as ResNet-18

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NETWORK LAYER PARAMETER DETAILS OF KAM-NET WHILE INFERENCE

Layer	Input/Output Channel	Kernel	Parameter
DW1	(3,3)	3×3	27
PW1	(3,8)	1×1	32
TA1	(8,8)	/	9
DW2	(8,8)	3×3	72
PW2	(8,16)	1×1	144
TA2	(16,16)	/	9
DW3	(16,16)	3×3	144
PW3	(16,24)	1×1	408
DW4	(24,24)	3×3	216
PW4	(24,1)	1×1	25
Total	/	/	1086

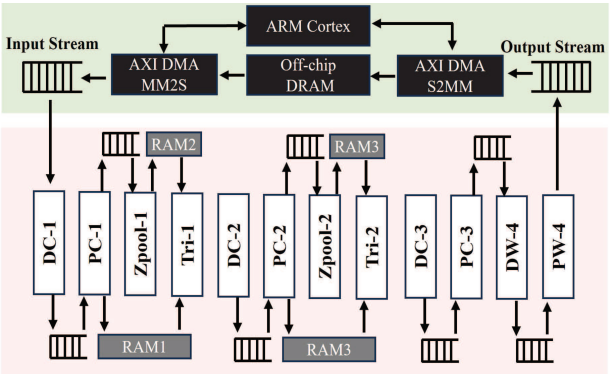


Fig. 4. Architecture of KAM-Net hardware accelerator.

(11 MB) and VGG-16 (14 MB), the amount of parameters is reduced by several orders of magnitude. This allows the entire model to be stored within the on-chip BRAM of the FPGA, effectively reducing the latency caused by frequent data transfers between on-chip and off-chip memory. As a result, system real-time performance is improved, and design complexity is significantly reduced. For instance, the ZU9EG chip used in this work provides approximately 4 MB of on-chip BRAM [21], which is insufficient to store the multi-megabyte parameters of typical models. However, it can easily accommodate the kilobyte-scale parameters of KAM-Net.

III. HARDWARE ARCHITECTURE DESIGN

The proposed architecture, tailored for the KAM-Net network, is depicted in Fig. 4. The green segment houses the PS with an ARM processor and DRAM, responsible for data transmission. The pink portion represents the programmable logic (PL) component, executing all calculation processes within KAM-Net. Modular modules handle deep convolution, point-wise convolution, and TA calculation, achieving spatial parallelism at multiple levels.

To illustrate the internal synchronization of the KAM-Net accelerator, Fig. 5 presents the pipelined processing of two consecutive images. Each module begins computation as soon as sufficient input is available, and partial outputs are streamed to the next stage. The three HFTA branches operate in parallel and are synchronized internally. FIFO buffers between

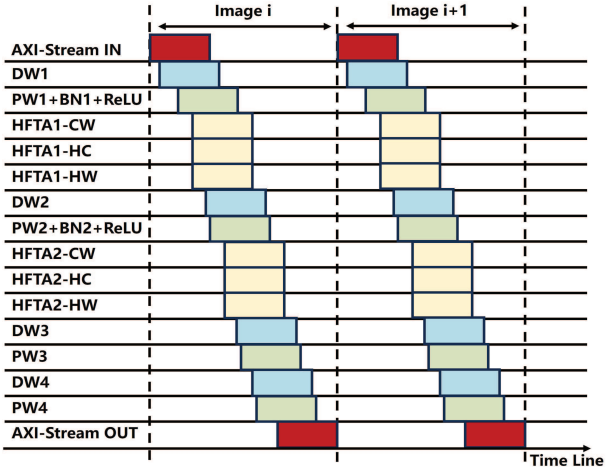


Fig. 5. Synchronization mechanism of the proposed architecture of the KAM-Net hardware accelerator.

modules ensure timing alignment and minimize pipeline stalls. Due to the use of a single AXI-Stream interface for both input and output, full pipelining is achieved within each image, but interframe concurrence is limited.

In addition, LUTRAM and BRAM store all weights and intermediate results, minimizing off-chip communication. To balance resource utilization and accuracy, 16-bit signed fixed-point numbers are used for data types. Section III-A details DSC blocks, while Section III-B focuses on hardware implementation of the TA mechanism.

A. DSC Hardware Accelerator

For depthwise convolution in DSC, we design a fundamental row-cache-based processing element (PE), as shown in Fig. 6(a), which incorporates two line buffers and nine input registers within each 3 × 3 convolution window. Each register corresponds to a fixed input 3 × 3 convolution weight, and the multiplication results pass through an addition tree. The registers are grouped into three rows, with data shifting from left to right after each clock cycle to simulate the sliding input effect. When reaching row or column boundaries, special consideration is required to avoid invalid outputs, as depicted in Fig. 6(b). To address this, we augment each multiplier calculation with an additional input set to 0, ensuring accurate results despite zero padding in convolution. The control logic for the multiplier inputs is detailed in the accompanying table.

For clarity, as shown in Table II, let $R1$, $R2$, and so on represent nine input registers, with the input image's dimensions denoted by W and H. In the current clock cycle, register $R5$ is at position (c, r) within the input image, where c signifies the column number and r denotes the row number. Referring to Fig. 6(b), at this stage, $(c = W, r < H)$. Consequently, the values of $R3$, $R6$, and $R9$ input to the multipliers are set to 0, ensuring accurate output data flow.

The above module serves as an input channel processing element. Since each input channel undergoes a separate 3 × 3 convolution, there is no interdependence across channels, allowing parallelization of processing elements equivalent to the number of input channels.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

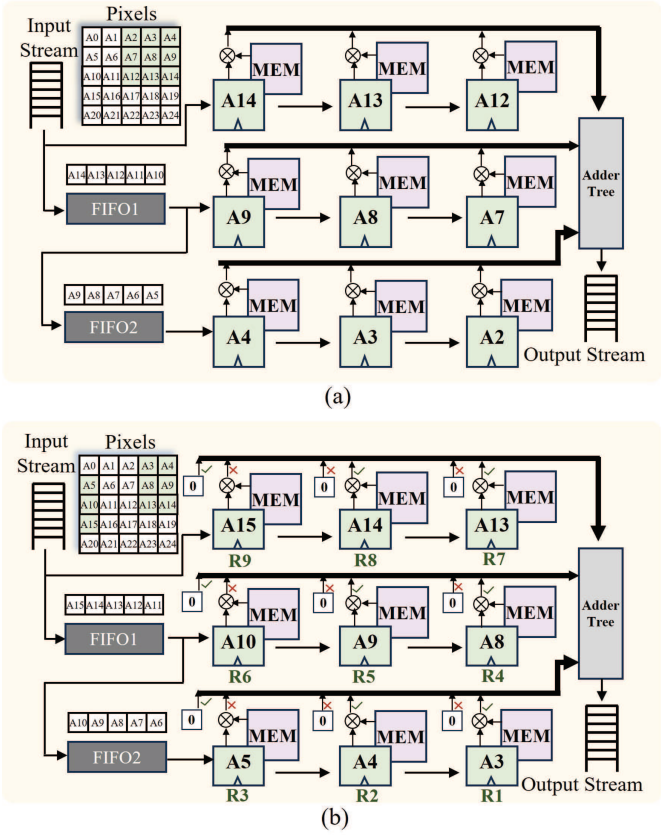


Fig. 6. Scheme of pipeline based DSC accelerator. (a) Convolution hardware circuit based on line buffer. (b) Convolution hardware circuit with padding support.

TABLE II
DETERMINATION OF THE RESULTS OF THE CONVOLUTION OPERATOR AT DIFFERENT POSITIONS

Conditions	R1	R2	R3	R4	R5	R6	R7	R8	R9
$(c=0, r=0)$	0	0	0	0	R5	R6	0	R8	R9
$(c=W, r=0)$	0	0	0	R4	R5	0	R7	R8	0
$(c=0, r=H)$	0	R2	R3	0	R5	R6	0	0	0
$(c=W, r=H)$	R1	R2	0	R4	R5	0	0	0	0
$(c<W, r=0)$	0	0	0	R4	R5	R6	R7	R8	R9
$(c<W, r=H)$	R1	R2	R3	R4	R5	R6	0	0	0
$(c=0, r<H)$	0	R2	R3	0	R5	R6	0	R8	R9
$(c=W, r<H)$	R1	R2	0	R4	R5	0	R7	R8	0
else	R1	R2	R3	R4	R5	R6	R7	R8	R9

The calculation module for the pointwise convolution in DSC is relatively simple. With a 1×1 convolution window, we can directly perform multiplication and accumulation calculations for each data stream without further complexity.

B. TA Hardware Accelerator

The lightweight TA mechanism operates along the $(C, W), (H, C)$, and (H, W) axes, applying Zpool operations independently in each direction. After convolving the resulting maximum and average values, attention weights are computed and multiplied with input pixels, followed by averaging. TA PEs are designed for each direction to enable parallel computation.

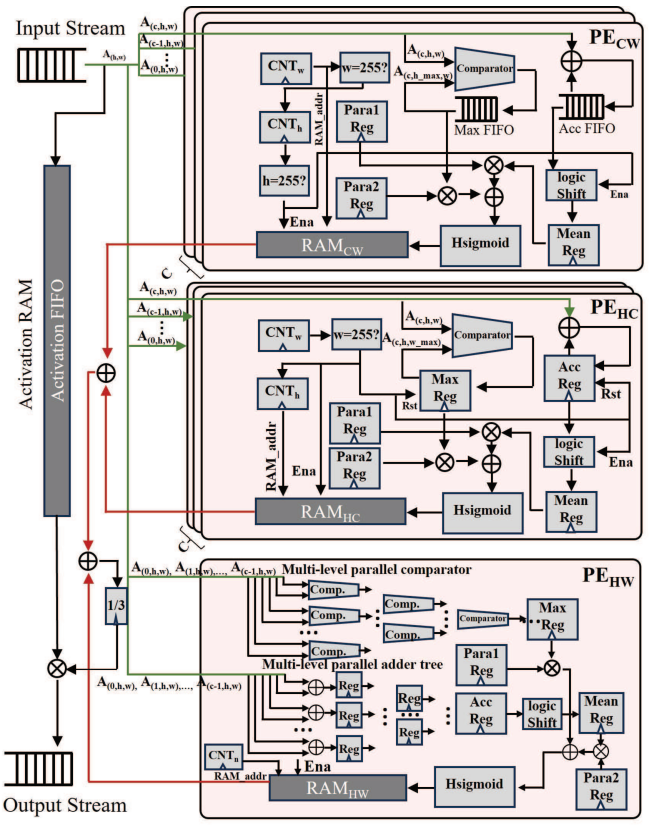


Fig. 7. Scheme of the proposed HFTA accelerator.

As shown in Fig. 7, the input stream accommodates $C \times N$ in all directions (where $N = 16$ denotes the bit width of each data), ensuring that one pixel is processed per clock cycle. For the (C, W) direction, maximum and average values are computed for each column, stored temporarily in Max FIFO and Acc FIFO. Once filled, the FIFOs extract data for processing. After processing all image data, maximum and accumulated value vectors are cached in the FIFO, and averages are derived through logical right shift.

To control RAM_{CW}, CNT_w and CNT_h track incoming data coordinates. RAM operation begins upon reaching the final row count, storing attention weights based on CNT_w. PE_{CW} is instantiated in parallel, facilitating concurrent weight calculation.

Notably, since data flow row-wise, there is no need for FIFOs to temporarily store maximum and accumulated values in PE_{HC}. We employ two registers for real-time max value and mean value calculations instead of FIFOs. Similarly with PE_{CW}, RAM_{HC} is controlled by counters, storing weights based on CNT_h. PE_{HC} computes weights simultaneously.

In the (H, W) direction, all channel data are inputted directly into comparators and addition trees. PE_{HW} calculates weights using CNT_n as RAM address.

The design of the comparator follows the principle of signed fixed-point number comparison in FPGA. First, the size is judged based on the first digit (positive or negative). If the first digit is equal, the size is judged step by step from large to small based on the remaining digits.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

After computing weights across directions, fusion involves multiplying and accumulating them with input pixels, followed by averaging. The activation FIFO temporarily stores input stream data, which is then multiplied with computed weights, concluding the TA calculation process.

IV. Experiments and Results

To verify the performance of algorithm/hardware codesign, we will conduct algorithm verification and comparison in Section IV-A and illustrate hardware performance in Section IV-B.

A. KAM-Net Algorithm Evaluation

1) Parameters Setting: The batch size set in the experiment is 32, the initial learning rate is 0.005, the learning rate decays by 0.95 every ten epochs, the optimizer is Adam, and the loss function is the sum of the Dice loss function and the BCE function commonly used in segmentation tasks. Except for the proposed KAM-Net, all general semantic segmentation models used for comparison are implemented using the open-source toolbox Segmentation Models Pytorch (SMP) library [22].

2) Computation Platform: We implement all the methods on PyCharm with the open-source toolbox Pytorch 1.11 [23]. We run our method in a NVIDIA RTX 2070 super (with 8-GB memory) on Windows 11 with Intel i9-12900H.

3) Evaluation Metrics: To ensure fair comparison across all benchmarked methods, we adopt a unified pixel-wise evaluation protocol. Specifically, all model outputs are first normalized using a sigmoid activation, producing values in the range of $[0, 1]$. A fixed threshold of 0.5 is then applied to obtain binary masks: pixels with predicted values ≥ 0.5 are classified as foreground (defect) and those ≤ 0.5 as background. This binarization rule, together with the ground truth, allows for consistent determination of true positives (TP), false positives (FP), and false negatives (FN). After that, the mean intersection over union (mIoU), $F1$-score, and sensitivity (also called recall) are used to evaluate the prediction performance. The mathematical definitions are shown in (3)–(5). Note that both the $F1$-score and sensitivity are computed for the defect class (class 0), while the mIoU represents the average IoU over all classes (in binary classification, k is 1)

$$\text{mIOU} = \frac{1}{k+1}\sum^{k}\frac{\text{TP}}{\text{FN}+\text{FP}+\text{TP}} \tag{3}$$

$$F1 = \frac{2\text{TP}}{2\text{TP}+\text{FN}+\text{FP}} \tag{4}$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP}+\text{FN}}. \tag{5}$$

We also use the number of parameters and frames per second (FPS) to demonstrate the real-time capability of the proposed approach.

4) Datasets' Description: This article first demonstrates the effectiveness of the proposed method using a collected Glass dataset. In addition, three surface defect datasets are used to demonstrate and evaluate the applicability and generality of the proposed method, including DAGM 2007 Defect dataset, MT Defect dataset, and Road Defect dataset.

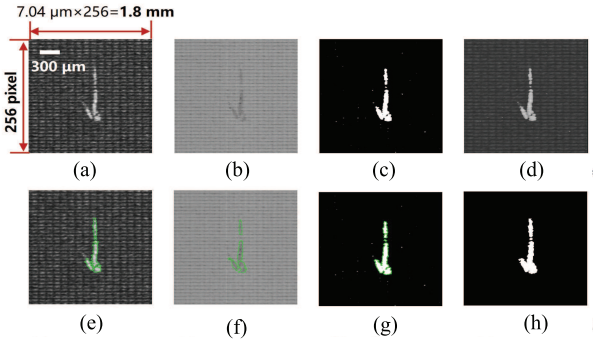


Fig. 8. Collected glass defect dataset with semantic annotation. (a) Reflect image. (b) Dark image. (c) Bright image. (d) Fusion image. (e) Reflect label. (f) Dark label. (g) Bright label. (h) Fusion label.

a) Glass surface defect dataset: The Glass dataset utilized in this study originates from a real glass production line [Fig. 1(b)], where flat glass continuously moves through the inspection equipment via rollers. This equipment comprises a light source and a camera, with three distinct light sources positioned strategically: reflected field above, dark field below, and bright field below the glass line. Each light source corresponds to a line-scan camera for synchronized acquisition. The system uses three identical line-scan cameras (8192×1 pixels, 12-bit depth, and 7.04 μm pixel size) operating at a line rate of 13 kHz. The cameras are mounted at fixed intervals with slightly different angles to ensure full coverage of the glass surface. As the substrate moves at a constant speed beneath the cameras, 2-D images are formed by stacking consecutive line scans.

To ensure that the image patches captured from the three cameras (with different viewing angles) correspond to the same physical region on the glass surface, a hardware-based alignment strategy is adopted. All cameras are mechanically calibrated using a reference target so that their fields of view overlap spatially. Additionally, they are synchronized via a shared encoder-trigger system, ensuring that each line scan corresponds to the same position along the moving direction. With this setup, the captured images are naturally aligned.

For the glass surface defect dataset used in our study, representative defective regions were extracted from the original high-resolution images and cropped into 256×256 patches to construct the dataset. To leverage the collected image information effectively, defect samples are labeled and synthesized. Initially, areas of glass defects are manually identified, and 256×256 image blocks centered on these areas are extracted [Fig. 8(a)–(c)], followed by semantic defect labeling on the three field images separately [Fig. 8(e)–(g)]. Annotations from the three field images are then combined into a synthesized defect annotation [Fig. 8(h)]. For images taken under three light sources, we directly concatenate them at the channel level [Fig. 8(d)]. In total, 1036 annotated images are produced, with 897 (87%) allocated to the training set and 139 (13%) to the test set.

b) Road defect dataset: This dataset contains two classes (crack and inlaid patch). The number of crack images is 500 with a size of around 2000×1500 pixels, which was collected

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

TABLE III
KAM-NET ABLATION EXPERIMENT ON GLASS SURFACE DEFECT DATASET

DSC	Reparameterization	TA	HFTA	mIOU	F1-score
✓				0.894	0.882
✓	✓			0.915	0.907
✓	✓	✓		0.925	0.919
✓	✓		✓	0.923	0.915

in [24]. The inlaid patch images that we collect by CCD contain 800 images of size around 3000 × 2000. Each defect image corresponds to a pixel-level label with different indexes. In this experiment, these raw images are randomly cut to 256 × 256 to improve the diversity of the dataset.

c) DAGM 2007 dataset: This artificially generated dataset [25] represents defects on textured backgrounds and closely approximates the real world. The dataset contains many classes of defects, and each raw image has a resolution of 512 × 512. In the label image of DAGM 2007, the defective area is roughly covered by an ellipse. In this experiment, we will only detect background and defects, without involving the specific classification of multiple defects. Furthermore, to fit the model, we scale the original image to 256 × 256.

d) MT defect dataset: The magnetic-tile defect dataset is presented in [26], which contains 1344 defect images, and each raw defect image corresponds to a pixel-level label. MTdefect dataset includes five types of defects: uneven, fray, crack, blowhole, and break, all these defect images with different resolutions. Most of these defect images contain a series of noise, e.g., the diversity of defect shape, complexity of texture, and the change of illumination intensity, all these factors bring a big challenge to detection. Similar to DAGM, in this experiment, we only detect background and defects and scale the image to 256 × 256.

5) Evaluation on Glass Defect Dataset: Table III performs ablation experiments on the three modules. The first column is the basic model using only DSC, and its mIOU and $F1$-score are both below 90%. As illustrated in Section II, the reparameterization was first introduced to the basic model, and both indicators achieved significant improvements, reaching more than 90%. In the third row, we continued to add the TA module, mIOU reached 92.5%, and $F1$-score reached 91.9%, which is the highest level among the three sets of experiments. This shows that the modules adopted to our model can improve the performance of the model. It is worth mentioning that after making hardware-friendly improvements to TA, as shown in the fourth row, mIOU and $F1$-score did not drop significantly, which shows that our improvements are effective.

To evaluate the effectiveness of KAM-Net, we compare it with several widely used semantic segmentation models, including UNet, FPN, PSPNet, MANet, and LinkNet, all of which adopt ResNet-18 as the encoder. As shown in Table IV, KAM-Net achieves the best performance in terms of mIOU (0.923) and $F1$-score (0.915) while maintaining a competitive sensitivity (0.922). Notably, KAM-Net requires only 0.004 MB of parameters and reaches 1580 FPS, significantly outperforming all baselines in terms of model compactness and inference speed.

TABLE IV
COMPARISON WITH EXISTING GENERAL SEMANTIC SEGMENTATION MODELS ON GLASS DATASET

Model	mIOU	F1	Sens.	Param	GFlops	FPS
UNet [27]	0.914	0.906	0.908	14MB	5.40	248
FPN [28]	0.909	0.901	0.912	13MB	4.44	333
PSPNet [29]	0.893	0.880	0.890	0.836MB	1.43	366
MANet [30]	0.917	0.910	**0.928**	22MB	5.90	290
LinkNet [31]	0.914	0.906	0.917	12MB	3.02	414
Proposed	**0.923**	**0.915**	0.922	**0.004MB**	**0.078**	**1580**

* Except the proposed KAM-Net, all other general models use ResNet-18 as encoder.

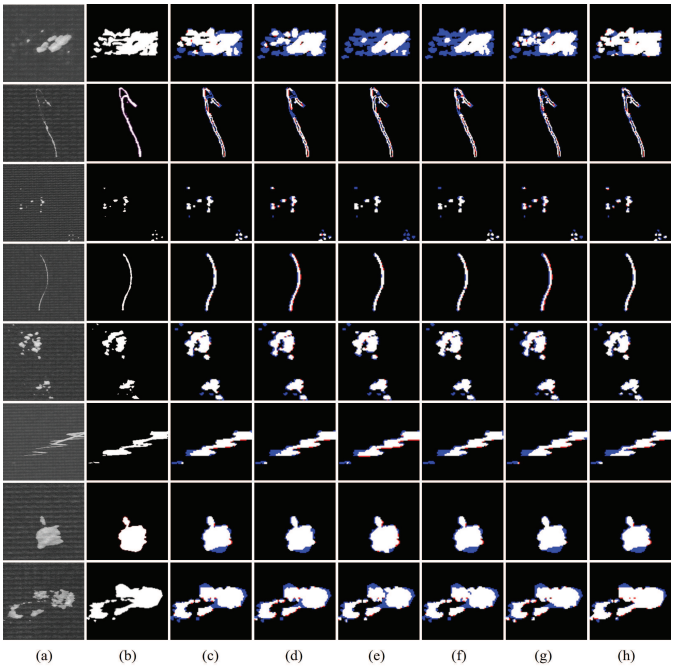


Fig. 9. Comparison of segmentation results on the Glass dataset. Pixels that are wrongly predicted as defects are marked in red, pixels that are predicted missing are marked in blue, and pixels that are predicted as normal defects are white. (a) Original image. (b) Ground truth. (c) UNet. (d) FPN. (e) PSPNet. (f) MANet. (g) LinkNet. (h) KAM-Net.

To evaluate the applicability of KAM-Net across different defect types, we conducted experiments on typical surface anomalies such as bubbles, scratches, and inclusions collected from real glass production lines. As shown in Fig. 9, the model performs well on both point-like and locally clustered defects. However, it shows limited sensitivity to elongated linear defects, potentially due to the absence of pooling layers and the resulting restricted receptive field. This tradeoff, though beneficial for maintaining spatial resolution and hardware efficiency, may impair long-range continuity modeling.

B. Generalization Evaluation on Public Datasets

To further improve segmentation performance on public datasets while preserving the lightweight nature of KAM-Net, we adopt a deeper configuration with six KAM blocks. This modification enhances the network's feature representation ability without significantly increasing model size and demonstrates the scalability of our architecture across various tasks.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

TABLE V

COMPARISON WITH EXISTING GENERAL SEMANTIC SEGMENTATION MODELS ON COMMON DATASET

Models	Road Defect [24]		DAGM [25]		Tile Defect [26]	
	mIOU	F1	mIOU	F1	mIOU	F1
UNet [27]	0.701	0.655	0.719	0.599	0.593	0.414
FPN [28]	**0.712**	**0.674**	0.654	0.458	0.604	0.432
PSPNet [29]	0.711	0.673	**0.754**	**0.664**	0.616	0.471
MANet [30]	0.673	0.615	0.724	0.602	0.610	0.451
LinkNet [31]	0.666	0.595	0.694	0.538	0.607	0.453
Proposed	0.705	0.663	0.732	0.627	**0.705**	**0.628**

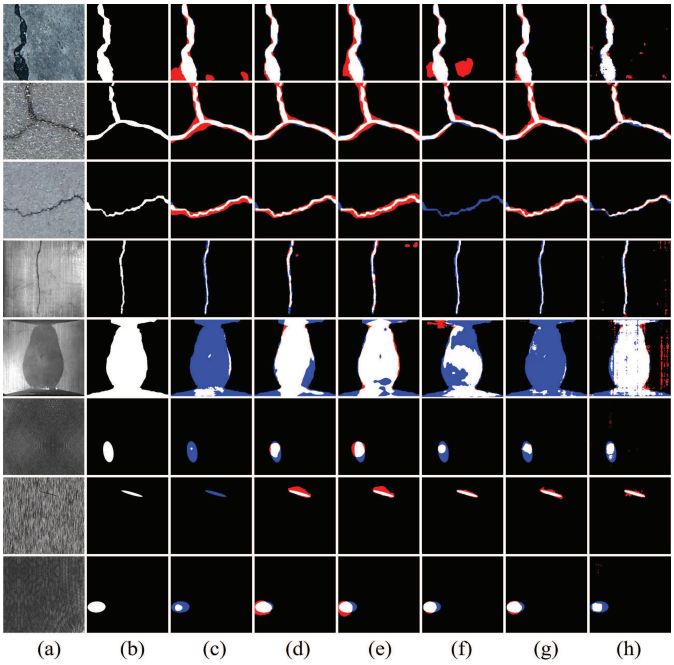


Fig. 10. Comparison of segmentation results on the Common dataset. The first three rows of images are from the Road Defect dataset. The middle two rows of images are from the Tile Defect dataset, and the last three rows of images are from the DAGM dataset. Pixels that are wrongly predicted as defects are marked in red, pixels that are predicted missing are marked in blue, and pixels that are predicted as normal defects are white. (a) Original image. (b) Ground truth. (c) UNet. (d) FPN. (e) PSPNet. (f) MANet. (g) LinkNet. (h) KAM-Net.

As demonstrated in Table V, our algorithm was evaluated on publicly available semantic segmentation datasets. The results indicate that our algorithm achieved the highest mIOU and $F1$-score on the Tile Defect dataset. Furthermore, our algorithm also exhibited competitive performance on the other two datasets. Given that our model has significantly fewer parameters and computational requirements compared to other models, these findings underscore the potential of our model for further application in other image segmentation domains.

Fig. 10 illustrates the visual segmentation results of KAM-Net and other competing models across three public datasets. Overall, KAM-Net successfully captures the primary contours of defect regions. However, it occasionally produces dispersed FP pixels, particularly in background or low-texture areas. This issue is primarily attributed to the absence of pooling layers, which restricts the network's ability to model global contextual dependencies. Furthermore, the lack of hierarchical abstraction makes the model overly sensitive to local contrast variations, resulting in spurious activations around defect boundaries or within textured backgrounds.

TABLE VI

RESOURCE USAGE FOR CNN ACCELERATOR WORKS ON FPGA

	[32]	[33]	[34]	Ours
FPGA	Stratix V GXA7	Stratix 10 NX	Z7045	ZU9EG
Network	ResNet-18	ResNet-18	ResNet-18	KAM-Net
Image Size	224×224×3	224×224×3	224×224×3	256×256×3
Frequency (MHz)	127	300	140	200
Precision (bit)	mixed (8A/1-4W)	8	16	16
LUTs	332k(35%)	1407k(67%)	88k(40%)	53k(19%)
DSPs	0(0%)	3960(51%)	864(96%)	1289(51%)
BRAMs(Xilinx) M20K(Intel)	1762(69%)	7000(98%)	320(59%)	829.5(91%)
FPS	245.2	4174	6.6	2564

C. Hardware Evaluation on FPGA

Our hardware platform employs the Xilinx ZCU102 development board, featuring the ZYNQ Ultrascale+ ZU9EG as its core computing chip, which integrates ARM and FPGA components. As elucidated in Section III, we leverage the ARM processor (PS side) for storage control, while the FPGA (PL side) handles model inference computations. The frequency of the PL part is running at 200 MHz, which is provided by the PS internal clock. Resource utilization statistics for our model are detailed in Table VI. To optimize throughput, we utilize on-chip BRAM to fully store model parameters and intermediate values, resulting in significant BRAM resource consumption, reaching 91.0%. Furthermore, our utilization of 16-bit fixed-point numbers for inference necessitates on-chip DSPs for each multiplication step, leading to the design of convolution calculation modules with high parallelism levels and substantial DSP resource utilization. Our hardware architecture demonstrates modest consumption of on-chip LUTs, utilizing only 19% of available resources.

To further evaluate the deployment efficiency of KAM-Net, Table VI compares its hardware resource utilization with several published FPGA implementations of ResNet-18. This comparison is reasonable because most baseline segmentation models in Table IV (e.g., UNet, FPN, and PSPNet) adopt ResNet-18 as their encoder backbone. The table reports key metrics, including device type, frequency, precision, resource consumption (LUTs, DSPs, and BRAMs), and FPS. As shown, KAM-Net achieves a favorable tradeoff between accuracy and efficiency, utilizing only 19% LUTs and 51% DSPs while delivering 2564 FPS, demonstrating strong hardware-friendliness and deployment potential.

Nevertheless, we acknowledge that the BRAM usage reaches 91%, which may limit the scalability and upgradability of the system. This high BRAM demand primarily stems from the column-wise computation in the proposed TA module. To support pipelined execution, our current design buffers the entire feature map for the column-direction attention, in contrast to the row- and channel-wise paths, which require only single-line buffers. For example, buffering a 256 × 256 feature map with 16-bit precision requires approximately 30

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

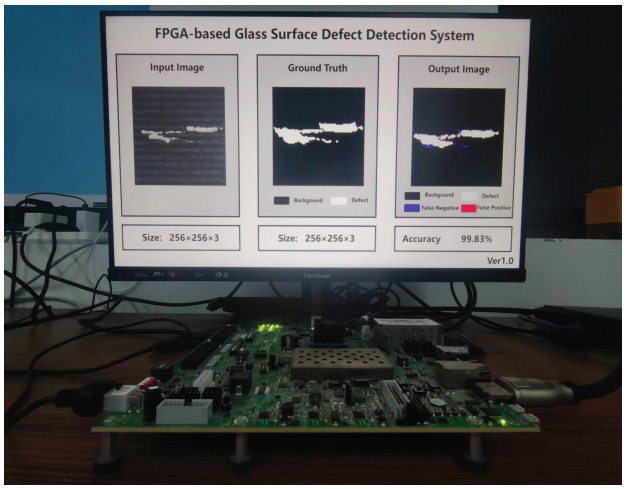


Fig. 11. FPGA-based glass surface defect detection hardware system.

TABLE VII
SPEED AND POWER COMPARISON OF KAM-NET BETWEEN DIFFERENT TYPES OF HARDWARE

Architecture	Device	Speed (ms)	FPS	power(W)
CPU	Intel I9-12900H	25.18	40	176
GPU	Nvidia RTX 2070 super	0.63	1580	72
FPGA	Xilinx ZU9EG	**0.39**	**2564**	**18**

BRAMs (36 kb each) for a single channel, and the consumption increases with the number of channels.

To address this issue, one potential solution is to utilize the PL-side DDR memory to transpose the feature map externally, allowing the column-wise path to be processed in a line-buffered manner similar to the row-wise path. Although this approach requires more complex memory control and pipeline reconfiguration, we consider it a promising direction for future work and have outlined this optimization opportunity in the conclusion section.

The implementation of KAM-Net on FPGA is depicted in Fig. 11. The FPGA image and its corresponding ground truth are dynamically loaded from the ARM side DDR of the SoC to the screen in real time. In the displayed ground truth, defective pixels are represented in white against a black background. Simultaneously, the processing results generated by the FPGA and stored in the ARM-side DDR are presented on the screen. In these results, misdetected pixels are highlighted in blue, while missed pixels are marked in red, aiding in comparison and inspection. The displayed processing results demonstrate that the FPGA effectively identifies the position of defective pixels as indicated in the ground truth. The pixel classification accuracy in the current image depicted in Fig. 11 stands at 99.83%.

We conducted speed and power consumption tests on three platforms—CPU, GPU, and FPGA—while running our model on the glass defect detection dataset. Results are summarized in Table VII. Due to the inherent limitations of CPUs in handling massively parallel computations, they exhibited the slowest processing speeds. GPUs, on the other hand, demonstrated a higher processing speed of 1580 FPS, albeit with significantly larger power consumption. In contrast, the FPGA achieved a processing speed of 2564 FPS while consuming only 18 W of power, surpassing the GPU's performance. This superior performance can be attributed to the pipeline processing technology implemented in the FPGA, enabling true parallel processing of images while simultaneously inputting and computing, a capability beyond the reach of GPUs.

V. CONCLUSION

This article introduces KAM-Net, a kilobyte-scale ultralightweight neural network model tailored for glass surface defect segmentation, along with its efficient FPGA architecture. The key contribution of the algorithm lies in the development of the KAM block, which integrates depth-separable convolution, reparameterization, and TA mechanism. Through the stacking of KAM blocks, the model effectively delineates defective pixels with minimal parameterization and computational overhead. KAM-Net attains notable performance metrics, achieving an mIOU of 92.3% and $F1$-score of 91.5% on the glass defect detection dataset while demonstrating competitive performance on other common segmentation datasets. Moreover, an efficient depth-separable convolution hardware architecture is proposed for FPGA deployment, enhancing computational efficiency through near-zero latency stream processing. Notably, the hardware-friendly optimization of TA, implemented on FPGA for the first time, further contributes to real-time performance in glass surface defect segmentation. Specifically, the proposed architecture achieves a processing speed exceeding 2564 FPS for $256 \times 256 \times 3$ images, accompanied by a low power consumption of 18 W.

In terms of practical defect resolution, the system reliably detects surface anomalies as small as 8×8 pixels under a 7.04-μm/pixel imaging setting, corresponding to a physical size of approximately 56 μm. This highlights the model's capability for submillimeter defect detection while also pointing to future opportunities to improve the detection of ultrafine defects through high-resolution imaging systems. Additionally, to further enhance the scalability of the hardware design, future work will explore BRAM-efficient implementations of column-wise attention—such as leveraging off-chip DDR-based feature map transposition—to reduce on-chip memory usage without compromising streaming performance.

REFERENCES

[1] X. Zhou et al., "A surface defect detection framework for glass bottle bottom using visual attention model and wavelet transform," *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2189–2201, Apr. 2020.

[2] J. Jiang, P. Cao, Z. Lu, W. Lou, and Y. Yang, "Surface defect detection for mobile phone back glass based on symmetric convolutional neural network deep learning," *Appl. Sci.*, vol. 10, no. 10, p. 3621, May 2020.

[3] W. Yang, Y. Zhang, Y. Dong, D. Xu, and T. Pan, "Development of machine vision system for off-line inspection of fine defects on glass screen surface," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–8, 2022.

[4] A. Huang, Z. Cao, C. Wang, J. Wen, F. Lu, and L. Xu, "An FPGA-based on-chip neural network for TDLAS tomography in dynamic flames," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–11, 2021.

[5] M. Sahani and P. K. Dash, "Deep convolutional stack autoencoder of process adaptive VMD data with robust multikernel RVFLN for power quality events recognition," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–12, 2021.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.

[6] M. Abbasnezhad and B. Alizadeh, "FPGA-based implementation of an artificial neural network for measurement acceleration in BOTDA sensors," *IEEE Trans. Instrum. Meas.*, vol. 68, no. 11, pp. 4326–4334, Nov. 2019.
[7] S. Ran et al., "Homecare-oriented ECG diagnosis with large-scale deep neural network for continuous monitoring on embedded devices," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–13, 2022.
[8] X. Xu, Y. Chen, K. Zhu, J. Yang, Z. Tan, and M. Luo, "Research on FPGA pulse laser ranging method based on deep learning," *IEEE Trans. Instrum. Meas.*, vol. 70, pp. 1–11, 2021.
[9] S. Lucia, D. Navarro, Ó. Lucía, P. Zometa, and R. Findeisen, "Optimized FPGA implementation of model predictive control for embedded systems using high-level synthesis tool," *IEEE Trans. Ind. Informat.*, vol. 14, no. 1, pp. 137–145, Jan. 2018.
[10] P. Cížek and J. Faigl, "Real-time FPGA-based detection of speeded-up robust features using separable convolution," *IEEE Trans. Ind. Informat.*, vol. 14, no. 3, pp. 1155–1163, Mar. 2018.
[11] Y. Li et al., "A 34-FPS 698-GOP/s/W binarized deep neural network-based natural scene text interpretation accelerator for mobile edge computing," *IEEE Trans. Ind. Electron.*, vol. 66, no. 9, pp. 7407–7416, Sep. 2019.
[12] Y. Lyu, L. Bai, and X. Huang, "ChipNet: Real-time LiDAR processing for drivable region segmentation on an FPGA," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 5, pp. 1769–1779, May 2019.
[13] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
[14] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
[15] D. Misra, T. Nalamada, A. U. Arasanipalai, and Q. Hou, "Rotate to attend: Convolutional triplet attention module," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2021, pp. 3139–3148.
[16] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 807–814.
[17] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel, and A. Ranjan, "Mobileone: An improved one millisecond mobile backbone," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2023, pp. 7907–7917.
[18] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 13733–13742.
[19] A. Veit, M. Wilber, and S. Belongie, "Residual networks behave like ensembles of relatively shallow networks," in *Proc. Neural Inf. Process. Syst. (NIPS)*, Dec. 2016, pp. 550–558.
[20] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," 2021, *arXiv:2106.08295*.
[21] AMD.(2025). *Zynq UltraScale+ MPSoC Product Selection Guide, Xilinx (an AMD Company), XMP104 (v2.8)*. [Online]. Available: https://docs.amd.com/v/u/en-U.S./zynq-ultrascale-plus-product-selection-guide
[22] P. Iakubovskii. (2019). *Segmentation Models Pytorch*. [Online]. Available: https://github.com/qubvel/segmentation_models.pytorch
[23] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Proces. syst.*, Jan. 2019, p. 32.
[24] F. Yang, L. Zhang, S. Yu, D. Prokhorov, X. Mei, and H. Ling, "Feature pyramid and hierarchical boosting network for pavement crack detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 4, pp. 1525–1535, Apr. 2020.
[25] M. Wieler and T. Hahn. *Weakly Supervised Learning for Industrial Optical Inspection*. Accessed: Jun. 25, 2017. [Online]. Available: https://hci.iwr.uni-heidelberg.de/node/3616
[26] Y. Huang, C. Qiu, and K. Yuan, "Surface defect saliency of magnetic tile," *Vis. Comput.*, vol. 36, no. 1, pp. 85–96, Jan. 2020.
[27] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Proc. Med. Image Comput. Comput.-Assist. Intervent.*, in Lecture Notes in Computer Science, vol. 1, 2015, pp. 234–241.
[28] T. Y. Lin, P. Dollàr, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 2117–2125.
[29] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2881–2890.
[30] T. Fan, G. Wang, Y. Li, and H. Wang, "MA-net: A multi-scale attention network for liver and tumor segmentation," *IEEE Access*, vol. 8, pp. 179656–179665, 2020.
[31] A. Chaurasia and E. Culurciello, "LinkNet: Exploiting encoder representations for efficient semantic segmentation," in *Proc. IEEE Vis. Commun. Image Process. (VCIP)*, Dec. 2017, pp. 1–4.
[32] C. Latotzke, T. Ciesielski, and T. Gemmeke, "Design of high-throughput mixed-precision CNN accelerators on FPGA," in *Proc. 32nd Int. Conf. Field-Programmable Log. Appl. (FPL)*, Aug. 2022, pp. 358–365.
[33] M. Doumet, M. Stan, M. Hall, and V. Betz, "H2PIPE: High throughput CNN inference on FPGAs with high-bandwidth memory," in *Proc. 34th Int. Conf. Field-Programmable Log. Appl. (FPL)*, Sep. 2024, pp. 69–77.
[34] P. Meloni et al., "NEURA ghe: Exploiting CPU-FPGA synergies for efficient and flexible CNN inference acceleration on Zynq SoCs," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 11, no. 3, pp. 1–24, Sep. 2018.



Bingrui Zhao (Student Member, IEEE) received the B.S. degree in electronic information engineering from Northeastern University, Shenyang, China, in 2017, and the M.S. degree in circuit and system from Xiamen University, Xiamen, China, in 2020. He is currently pursuing the Ph.D. degree with the Department of Control Science and Engineering, Hunan University, Changsha, China.

His research interests include industrial defect detection, neural network compression, and hardware acceleration.



Yaonan Wang (Member, IEEE) received the B.S. degree in computer engineering from East China University of Technology, Nanchang, China, in 1981, and the M.S. and Ph.D. degrees in electrical engineering from Hunan University, Changsha, China, in 1990 and 1994, respectively.

He has been a Professor at Hunan University since 1995. His current research interests include robotics and computer vision systems for industrial applications.

Dr. Wang is a fellow of Chinese Academy of Engineering.



Hui Zhang (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in pattern recognition and intelligent system from Hunan University, Changsha, China, in 2004, 2007, and 2012, respectively.

He is currently a Professor with the School of Robotics, Hunan University. He was appointed the Changjiang (Yangtze River) Distinguished Professorship in 2024. His research interests include machine vision, image processing, and visual tracking.



Jiaxuan Liu received the B.S. degrees in electrical and computer engineering from Shaanxi University of Science and Technology, Xi'an, China, in 2020, and the M.Sc. degree from Changsha University of Science and Technology, Changsha, China, in 2023. He is currently pursuing the Ph.D. degree with the National Engineering Research Center, Hunan University, Changsha.

His current research interests include image processing, machine learning, and deep learning.



Jinzhou Zhang received the B.S. degree from Chennan College, Changsha University of Science and Technology, Changsha, China, in 2016. He is currently pursuing the M.S. degree in electronic and information engineering with Changsha University of Science and Technology.

His research interests include industrial defect detection and hardware acceleration for neural networks.

Authorized licensed use limited to: Guangdong Univ of Tech. Downloaded on October 14,2026 at 07:15:32 UTC from IEEE Xplore. Restrictions apply.